

Module 4

UVM Components

Learning objectives

- Build complete UVM agents with driver, monitor, and sequencer

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["UVM Agent Architecture"]

T2["UVM Driver Implementation"]

T1 --> T2

T3["UVM Monitor Implementation"]

T2 --> T3

Build complete UVM agents with driver, monitor, and sequencer

Overview

- This module covers the core UVM components used to build verification environments. You'll learn ho...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

subgraph ifc["simple_interface.v"]

V[valid/ready]

AD[addr / data]

RES[result]

end

Module 4: DUT structure and key interfaces (see common_dut/rtl/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ[sequence] --> SQR[sequencer]

SQR --> DRV[driver]

DRV --> DUT[(interface DUT)]

DUT --> MON[monitor]

MON --> AP[analysis port]

Module 4: UVM layers, agents, and checkers for this phase.

1. DUT and interface protocol

- ``module4/dut/interfaces/simple_interface.v`` —
valid/ready handshake, address/data/result buses
- Clocked, resettable interface models streaming transactions
- Agent maps one transaction type to one interface beat

2. Agent-centric UVM architecture

- Agent bundles driver, monitor, sequencer; env holds agent + scoreboard
- TLM analysis ports connect monitor → scoreboard (and optional coverage)
- ``InterfaceTransaction`` defines fields, ``copy`/`compare``, and constraints
- Complete path: sequence → sequencer → driver → DUT → monitor → scoreboard

3. Component responsibilities

- Driver: pull items from sequencer, drive pins per protocol
- Monitor: passive observation, broadcast transactions on analysis port
- Sequencer: arbitration and sequence execution
- Scoreboard: expected vs actual from monitor streams

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

ITEM[sequence_item] --> DRV[drive beat]

DRV --> DUT[(DUT)]

DUT --> MON[monitor item]

MON --> SB{scoreboard compare}

SB -->|match| PASS

Module 4: how tests, checks, and metrics close verification goals.

1. Integration testing method

- ``test_complete_agent.py`` — end-to-end agent, env, and test class
- Run: ``./scripts/module4.sh --pyuvm-tests`` or ``make TEST=test_complete_agent``
- Per-component examples (``driver_example.py``, ...) isolate behavior before integration

2. Checking strategy

- Scoreboard compares monitored results to predicted model or golden vectors
- Analysis port fan-out allows multiple checkers without changing monitor
- Sequences provide repeatable stimulus; random/constrained extensions in exercises

3. Closure

- Self-check via ``module4.sh --check``; demo screenshots per EXAMPLES.md section
- Assessment: drivers, monitors, sequencers, agents, scoreboards, TLM connections

Syllabus topics

1. UVM Agent Architecture (1/3)

- Agent Overview
- What is an agent?
- Agent components
- Agent purpose
- Agent types
- Active vs Passive Agents

1. UVM Agent Architecture (2/3)

- Active agents (driver + sequencer + monitor)
- Passive agents (monitor only)
- When to use each
- Agent configuration
- Agent Structure
- Driver component

1. UVM Agent Architecture (3/3)

- Monitor component
- Sequencer component
- Agent container

2. UVM Driver Implementation (1/4)

- Driver Overview
- Driver purpose
- Driver responsibilities
- Driver interface
- Driver lifecycle
- Driver Implementation

2. UVM Driver Implementation (2/4)

- Inheriting from ``uvm_driver``
- ``run_phase()` implementation
- Transaction reception
- Signal driving
- Driver-Sequencer Communication
- ``seq_item_port`` interface

2. UVM Driver Implementation (3/4)

- ``get_next_item()``
- ``item_done()``
- Transaction flow
- Signal-Level Driving
- DUT signal access
- Signal value assignment

2. UVM Driver Implementation (4/4)

- Timing control
- Protocol implementation

3. UVM Monitor Implementation (1/4)

- Monitor Overview
- Monitor purpose
- Monitor responsibilities
- Monitor types
- Monitor lifecycle
- Monitor Implementation

3. UVM Monitor Implementation (2/4)

- Inheriting from ``uvm_monitor``
- ``run_phase()` implementation
- Signal sampling
- Transaction creation
- Analysis Ports
- Analysis port purpose

3. UVM Monitor Implementation (3/4)

- Creating analysis ports
- Writing to analysis ports
- Analysis port connections
- Transaction Creation
- Sampling signals
- Creating transaction objects

3. UVM Monitor Implementation (4/4)

- Populating transaction fields
- Broadcasting transactions

4. UVM Sequencer and Sequences (1/5)

- Sequencer Overview
- Sequencer purpose
- Sequencer responsibilities
- Sequencer types
- Sequencer lifecycle
- Sequencer Implementation

4. UVM Sequencer and Sequences (2/5)

- Inheriting from ``uvm_sequencer``
- Default sequencer usage
- Custom sequencer features
- Sequencer configuration
- Sequence Items
- ``uvm_sequence_item`` base class

4. UVM Sequencer and Sequences (3/5)

- Transaction definition
- Transaction fields
- Transaction methods
- Sequence Basics
- ``uvm_sequence`` base class
- ``body()`` method

4. UVM Sequencer and Sequences (4/5)

- Sequence execution
- Sequence lifecycle
- Sequence Operations
- ``start_item()`
- ``finish_item()`
- ``wait_for_grant()`

4. UVM Sequencer and Sequences (5/5)

- Transaction creation

5. TLM (Transaction-Level Modeling) - Complete Coverage (1/11)

- TLM Overview
- What is TLM?
- TLM benefits
- TLM abstraction levels
- TLM communication patterns
- TLM Interface Types

5. TLM (Transaction-Level Modeling) - Complete Coverage (2/11)

- ``put`` interface - Unidirectional blocking put
- ``get`` interface - Unidirectional blocking get
- ``peek`` interface - Unidirectional non-blocking peek
- ``transport`` interface - Bidirectional blocking transport
- Interface characteristics and use cases
- TLM Port Types

5. TLM (Transaction-Level Modeling) - Complete Coverage (3/11)

- `uvm\_put\_port` - Put port
- `uvm\_get\_port` - Get port
- `uvm\_peek\_port` - Peek port
- `uvm\_transport\_port` - Transport port
- Port vs export vs implementation
- TLM Export Types

5. TLM (Transaction-Level Modeling) - Complete Coverage (4/11)

- `uvm\_put\_export` - Put export
- `uvm\_get\_export` - Get export
- `uvm\_peek\_export` - Peek export
- `uvm\_transport\_export` - Transport export
- Export usage patterns
- TLM Implementation Types

5. TLM (Transaction-Level Modeling) - Complete Coverage (5/11)

- ``uvm_put_imp`` - Put implementation
- ``uvm_get_imp`` - Get implementation
- ``uvm_peek_imp`` - Peek implementation
- ``uvm_transport_imp`` - Transport implementation
- Implementation requirements
- Analysis Ports and Exports (Special TLM Type)

5. TLM (Transaction-Level Modeling) - Complete Coverage (6/11)

- Analysis port concept
- Publisher-subscriber pattern
- Broadcast communication
- One-to-many connections
- ``uvm_analysis_port`` - Analysis port
- ``uvm_analysis_export`` - Analysis export

5. TLM (Transaction-Level Modeling) - Complete Coverage (7/11)

- ``uvm_analysis_imp`` - Analysis implementation
- Connection patterns
- TLM FIFOs
- ``uvm_tlm_fifo`` - TLM FIFO
- FIFO purpose and usage
- FIFO capacity and blocking behavior

5. TLM (Transaction-Level Modeling) - Complete Coverage (8/11)

- FIFO connection patterns
- When to use FIFOs
- TLM Connection Patterns
- Direct connections (port to export)
- FIFO connections (port to FIFO to export)
- Multi-port connections

5. TLM (Transaction-Level Modeling) - Complete Coverage (9/11)

- Hierarchical connections
- Connection best practices
- TLM Usage Patterns
- Producer-consumer pattern
- Request-response pattern
- Broadcast pattern

5. TLM (Transaction-Level Modeling) - Complete Coverage (10/11)

- Pipeline pattern
- TLM vs analysis ports
- TLM Implementation Examples
- Using put/get interfaces
- Using transport interface
- Using TLM FIFOs

5. TLM (Transaction-Level Modeling) - Complete Coverage (11/11)

- Combining TLM types
- TLM debugging

6. Scoreboard Implementation (1/4)

- Scoreboard Overview
- Scoreboard purpose
- Scoreboard types
- Scoreboard responsibilities
- Scoreboard lifecycle
- Scoreboard Implementation

6. Scoreboard Implementation (2/4)

- Inheriting from ``uvm_component``
- Analysis port connections
- Transaction storage
- Comparison logic
- Scoreboard Patterns
- Reference model comparison

6. Scoreboard Implementation (3/4)

- Expected vs actual
- Transaction matching
- Error reporting
- Advanced Scoreboards
- Multi-channel scoreboards
- Time-based matching

6. Scoreboard Implementation (4/4)

- Complex comparison logic
- Performance optimization

7. Transaction-Level Modeling (1/3)

- Transaction Concepts
- What are transactions?
- Transaction abstraction
- Transaction fields
- Transaction methods
- Transaction Design

7. Transaction-Level Modeling (2/3)

- Transaction class structure
- Field definition
- Constraint definition
- Method implementation
- Transaction Operations
- Transaction creation

7. Transaction-Level Modeling (3/3)

- Transaction copying
- Transaction comparison
- Transaction conversion

8. Complete Agent Example (1/3)

- Agent Structure
- Agent class definition
- Component instantiation
- Component connections
- Agent configuration
- Agent Build Phase

8. Complete Agent Example (2/3)

- Component creation
- Configuration application
- Active/passive selection
- Agent Connect Phase
- Driver-sequencer connection
- Monitor analysis port

8. Complete Agent Example (3/3)

- External connections

9. Sequence Libraries (1/2)

- Sequence Organization
- Base sequences
- Derived sequences
- Sequence libraries
- Sequence reuse
- Common Sequences

9. Sequence Libraries (2/2)

- Simple sequences
- Random sequences
- Constrained sequences
- Layered sequences

10. Agent Integration (1/2)

- Environment Integration
- Adding agents to environment
- Agent configuration
- Agent connections
- Agent coordination
- Test Integration

10. Agent Integration (2/2)

- Agent instantiation
- Sequence execution
- Test coordination
- Result checking

Hands-on examples

Module 4 self-check

```
$ ./scripts/module4.sh --check
```

Terminal: module4_check

```
#####  
Module 4: UVM Components#####  
#####
```

Usage: ./scripts/module4.sh [OPTIONS]

Module 4: UVM Components
This script runs examples and tests for Module 4.

OPTIONS:

Examples:

--drivers Run driver examples
--monitors Run monitor examples
--sequencers Run sequencer and sequence examples
--tlm Run TLM examples
--scoreboards Run scoreboard examples
--transactions Run transaction examples
--agents Run complete agent examples
--all-examples Run all examples (default)
--skip-examples Skip all examples

Tests:

--pyuvvm-tests Run pyuvvm tests

Environment:

--venv DIR Virtual environment directory (default: .venv)
--no-venv Don't use virtual environment
--sim SIMULATOR Simulator to use (default: verilator)

Other:

--help, -h Show this help message

EXAMPLES:

Run all examples
./scripts/module4.sh

Exercise scaffold

```
./scripts/module4.sh --scaffold
```

Demo: Driver Implementation

```
$ .venv/bin/python module4/examples/drivers/driver_example.py
```

Terminal: ex01_drivers

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module4/examples/drivers/
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Monitor Implementation

```
$ .venv/bin/python module4/examples/monitors/monitor_example.py
```

Terminal: ex02_monitors

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module4/examples/monitors
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Sequencer and Sequences

```
$ .venv/bin/python module4/examples/sequencers/sequencer_example.py
```

Terminal: ex03_sequencers

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module4/examples/sequence
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Complete Agent

```
$ .venv/bin/python module4/examples/agents/agent_example.py
```

Terminal: ex04_agents

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module4/examples/agents/a

from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Scoreboard Implementation

```
$ .venv/bin/python module4/examples/scoreboards/scoreboard_example.py
```

Terminal: ex05_scoreboards

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module4/examples/scoreboa
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: TLM Communication

```
$ .venv/bin/python module4/examples/tlm/tlm_example.py
```

Terminal: ex06_tlm

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module4/examples/tlm/tlm_
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Transaction Modeling

```
$ .venv/bin/python  
    module4/examples/transactions/transaction_example.py
```

Terminal: ex07_transactions

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module4/examples/transact
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Practice & assessment

Exercises

- Driver Implementation
- Monitor Implementation
- Sequence Creation
- Agent Building
- Scoreboard Implementation

Assessment checklist

- Understands agent architecture
- Can implement drivers
- Can implement monitors
- Can implement sequencers
- Can create sequences
- Can use analysis ports

Summary & next steps

- Build complete UVM agents with driver, monitor, and sequencer
- Complete module4/CHECKLIST.md
- Run `./scripts/module4.sh --check`
- Next: Next module in course